# VC Formal Lab
# Data Path Validation (DPV) App
## Setup and Standard Usage

### Learning Objectives

In this lab, you will use an ALU example to learn how to do the following:

- C to RTL equivalence verification using VC Formal DPV
- Set up tcl file to invoke VC Formal DPV
  - Set up spec and impl DUT import and configuration
  - Set up clocks and resets for RTL
  - Create lemmas to prove output equivalence
- Run proof & review results
- Debug failures using Verdi GUI to find bugs

**Lab Duration:**
**60 minutes**

## Lab Package Overview

In this lab, you will run VC Formal DPV to check an ALU with several arithmetic and logical operations. You will understand how to set up and run C2RTL equivalence checking using DPV, and how to debug and resolve the failures.

## Files Location

All files for this VC Formal lab are in directory:
$VC_STATIC_HOME/doc/vcst/examples/DPV/DPV_Basic/

| Directory Structure | |
|---|---|
| DPV_Basic | Lab main directory |
| README_VCFormal_DPV_Basic.pdf | Lab instructions |
| c/ | A behavioral implementation of ALU design in C/C++ |
| rtl/ | Synthesizable RTL code of the ALU design |
| run/ | Run directory |
| solution/ | Solution directory |

## Resources

The following resources are available for in-depth guidance regarding VC Formal usage, commands, and variables.

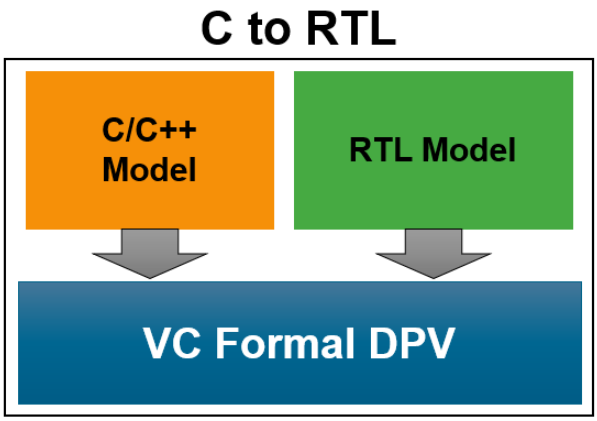
VC Formal and DPV User Guide:
$VC_STATIC_HOME/doc/vcst/VC_Formal_Docs/VC_Formal_UG.pdf
$VC_STATIC_HOME/doc/vcst/VC_Formal_Docs/VC_Formal_DPV_UG.pdf

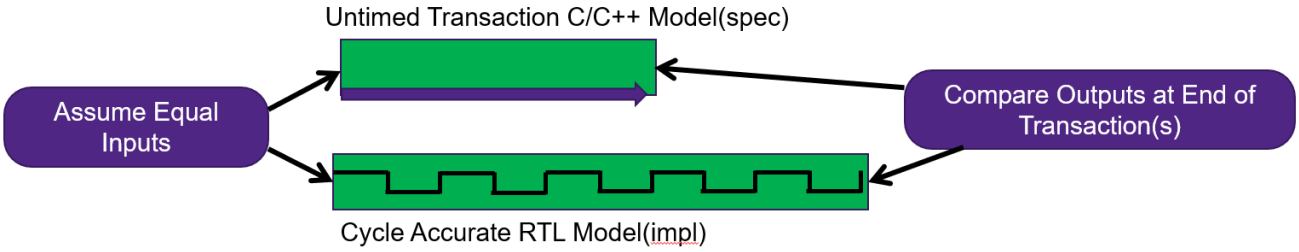
VC Formal Apps Quick References Guide:
$VC_STATIC_HOME/doc/vcst/VC_Formal_Docs/VC_Formal_Quick_Reference_Guide.pdf

VC Formal Apps Tcl Templates:
$VC_STATIC_HOME/doc/vcst/VC_Formal_Docs/Tcl_Templates/

## DUT Description

The objective of this lab is to introduce the verification methodology for equivalence checking of C/C++ model against RTL using VC Formal DPV. This generally requires having a C/C++ model of the architecture, which is considered as the golden model or the specification. After developing the RTL, i.e., the implementation, it is compared against the specification using VC Formal DPV.




The DUT is a simple ALU design with one latency from inputs to outputs, to support several arithmetic/logical operation modes with 2 inputs. It includes a C model for spec and an RTL design for impl, so users could use this Lab to practice several DPV convergence techniques to converge the proof.

### DUT in/out Port Description

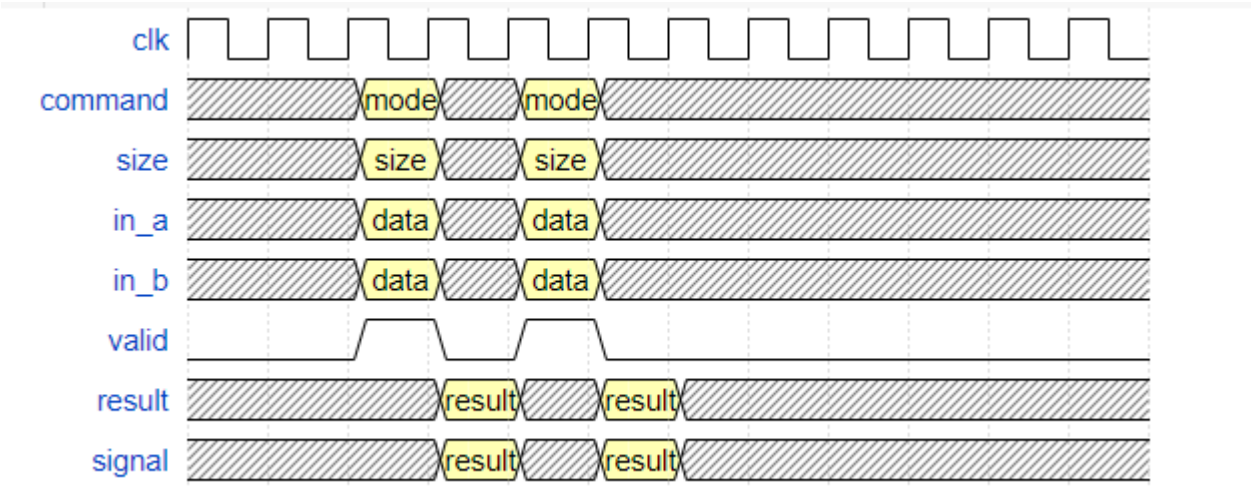
| Signal | Direction | Width | Description |
|---|---|---|---|
| clk | In | 1 | Clock |
| reset | In | 1 | Active high asynchronous reset signal |
| valid | In | 1 | Active high valid signal to indicate inputs are valid |
| command | In | 3 | Command for ALU operation mode<br>0: AND<br>1: OR<br>2: ADD |

| | | | |
|---|---|---|---|
| | | | 3: SUB<br>4: SATURATING ADD<br>5: SATURATING SUB<br>6: in_a×in_b 16×16 into 32 bits only<br>Please see details below * |
| size | In | 1 | Config operation bit width<br>0: 16-bit<br>1: 32-bit<br>No effect when command = 6 |
| in_a | In | 32 | Operand 1 of ALU input |
| in_b | In | 32 | Operand 2 of ALU input |
| result | Out | 32 | Output of ALU |
| signal | Out | 2 | Output flag for result status, see details below * |

*Operation mode detail of each command:

- 0 - AND (When result is all 1's then, signal[1] = 1. When result is all 0's then, signal[0] = 1)
- 1 - OR (When result is all 1's, then signal[1] = 1. When result is all 0's then, signal[0] = 1)
- 2 - ADD (inputs are 2's complement. When overflow, then signal[1] = 1. When result is zero, then signal[0] = 1)
- 3 - SUBTRACT (A-B, inputs are 2's complement. When overflow, then signal[1] = 1. When result is zero, then signal[0] = 1)
- 4 - SATURATING ADD (inputs are unsigned. When saturation, then signal[1] = 1 and result is all 1's to proper width. When result is zero, then signal[0] = 1)
- 5 - SATURATING SUBTRACT (A-B, inputs are unsigned. When saturation, then signal[1] = 1 and result is all 0's. Unsaturated zero result sets signal[0])
- 6 - A×B, 16×16 into 32 bits only, zero result sets signal[0]

**DUT Data in/out Timing Diagram**




## Prepare your Environment

1. Ensure that VC Formal and Verdi are set up in your terminal then change your working directory to run:

```
% cd run
```

Now you are ready to begin the lab.

## Create run.tcl file for DPV setup

VC Formal has a tcl based command interface. The most common way is to start with a tcl file to setup and compile the design.

The design files are under “c” and “rtl” directories.

2. Under run directory, create and open a file called run.tcl (any arbitrary name is ok to use) using an editor:

```
% vi run.tcl
```

3. Configure DPV default settings in run.tcl:

```
#Switch to DPV mode
set_fml_appmode DPV

#Must configure file host.qsub based on local environment
#set_host_file "host.qsub"
```

If you uncomment "#set_host_file …" to enable multi-processor support, edit file ./host.qsub to setup host details according to your local working environment. Refer to section 6.6 in document DPV_UserGuide.pdf for more information.

```
% vi host.qsub
```

```
#LSF example
1| | 2 | /tmp | LSF | bsub -q bnormal
#RSH example
1 |10.10.10.10 | 4 | /tmp | RSH | rsh
```

4. Enter following commands to compile spec, which is a C model:

```
proc compile_spec {} {
  create_design -name spec -top DPV_wrapper
  cppan -I. ../c/alu.cpp
  compile_design spec
}
```

5. Enter following commands to compile impl design, which is a Verilog RTL. The name of the top module is "alu":

```
proc compile_impl {} {
  create_design -name impl -top alu -clock clk -reset reset
## RTL lab case with bug injection
  vcs -sverilog ../rtl/multiplier_bug.sv ../rtl/alu_bug.sv
  compile_design impl
}
```

6. Define lemmas/assumes in the tcl procs as below, please note that this setup will exclude multiplication mode which is mode "6", to simplify the proof. Make sure to save your changes before going to step 7.

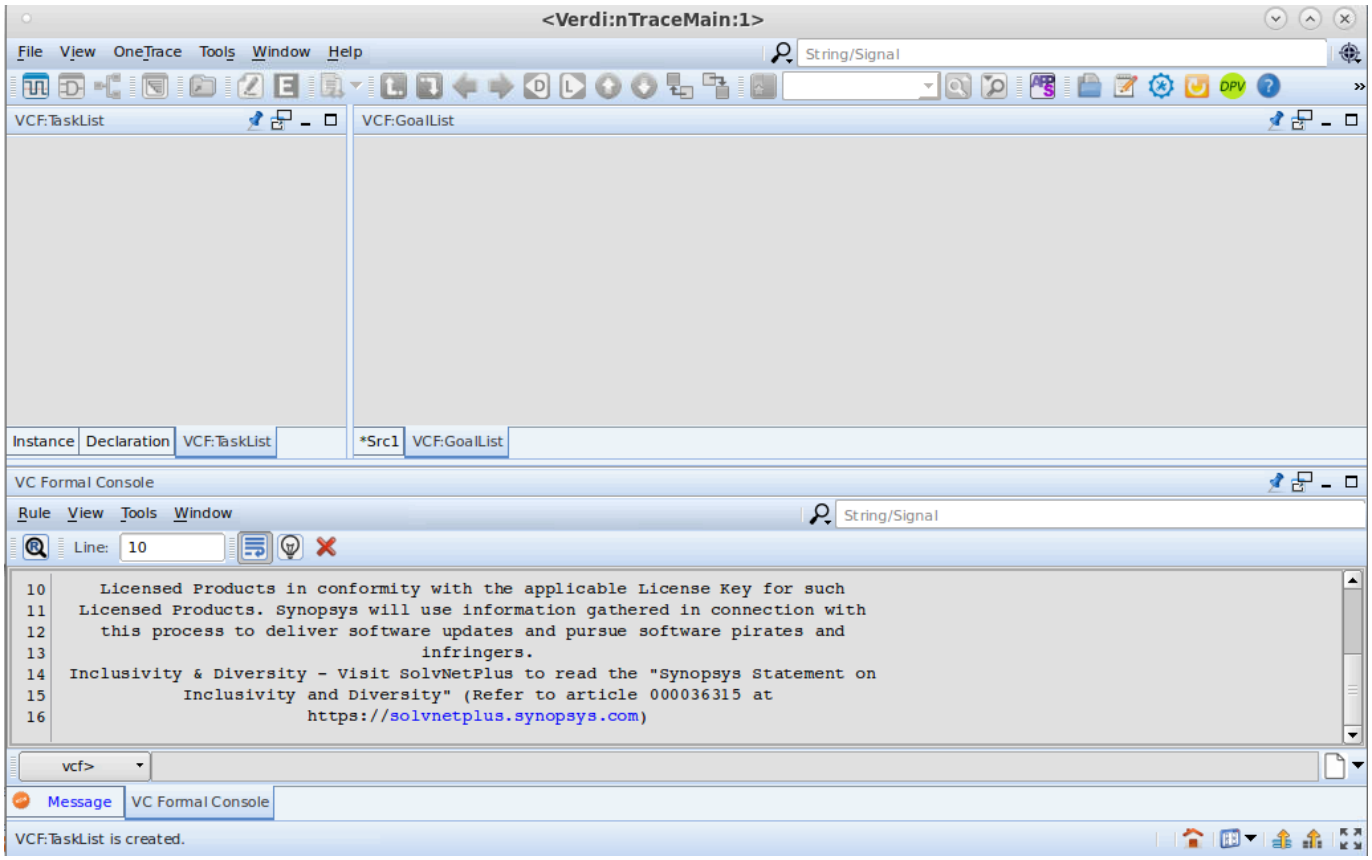
```
proc global_assumes {} {
  map_by_name -inputs -specphase 1 -implphase 1
  assume command_range = spec.command(1) < 6
}

proc ual {} {

  global_assumes

  lemma result_equal_small = impl.valid(1) && impl.size(1) == 0 ->
impl.result(3)[15:0] == spec.result(1)[15:0]
  lemma result_equal_big = impl.valid(1) && impl.size(1) == 1  ->
impl.result(3) == spec.result(1)
  lemma signal_equal = impl.valid(1) -> impl.signal(3)[1:0] ==
spec.signal(1)[1:0]
}

set_user_assumes_lemmas_procedure "ual"
```

## Start VC Formal in GUI mode

7. Start the tool in GUI mode:

```
% vcf -f run.tcl -gui
```

The GUI starts in the VC Formal mode, with icons, tables, tabs and windows especially designed for property verification.

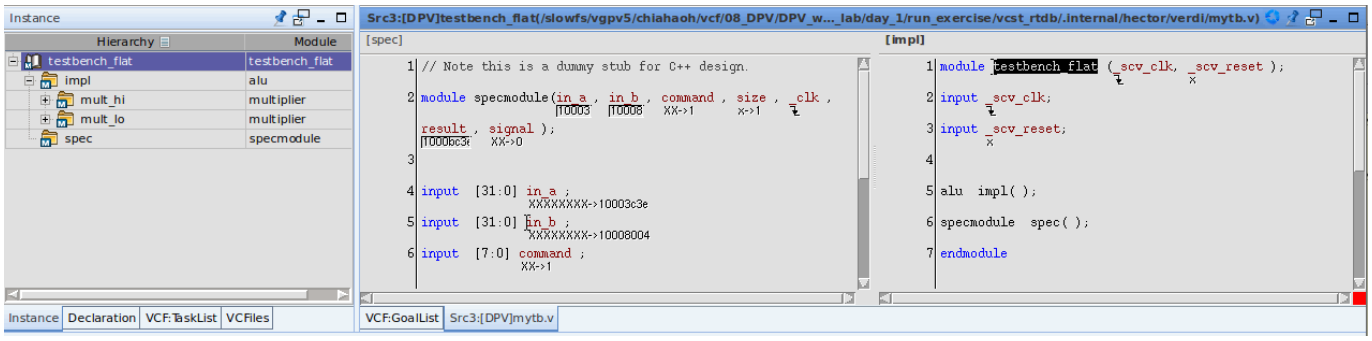



## Review Equivalence Checking Result

8. Use the following compile and compose commands for both spec and impl designs. It will create a DPV formal model, DFG (data flow graph) for both spec and impl designs.

```
vcf> compile_spec
vcf> compile_impl
vcf> compose
```

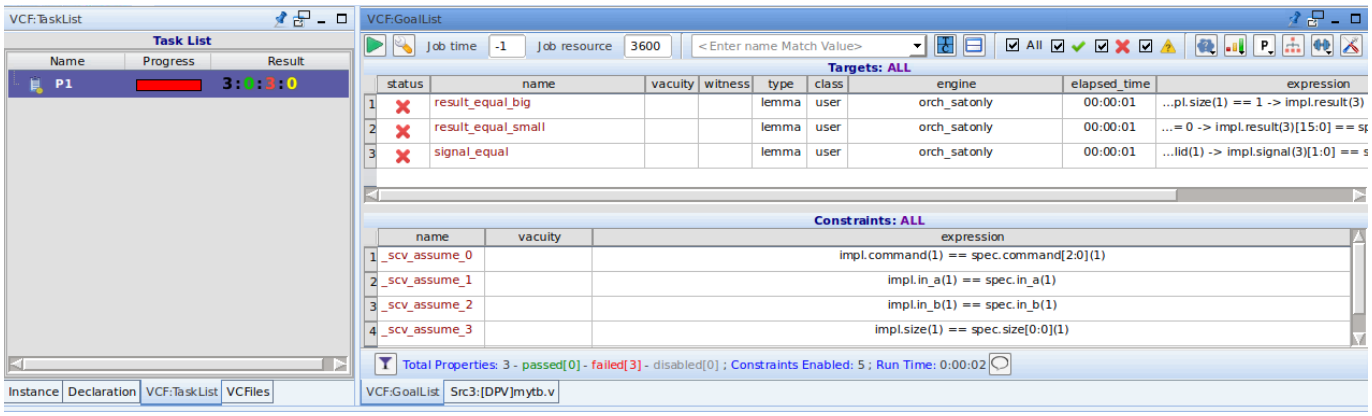
9. Review the result and make sure all DUT modules, including C and RTL, are imported completely, and compiled without any error.




10. Run solveNB command with the user assumes and lemmas proc ‘ual’ defined in step 6 to execute spec to impl transaction equivalence checking.

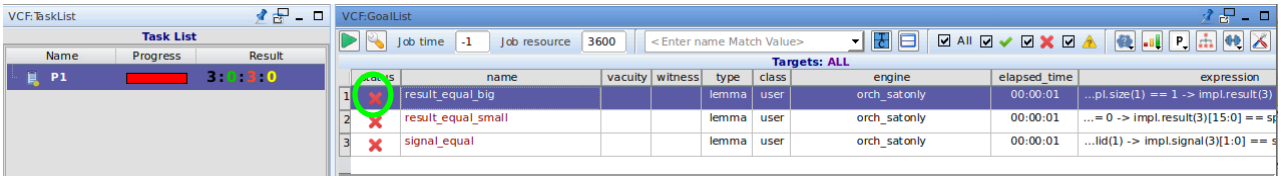
```
vcf> solveNB P1
```

Three lemmas have falsified result as shown below.

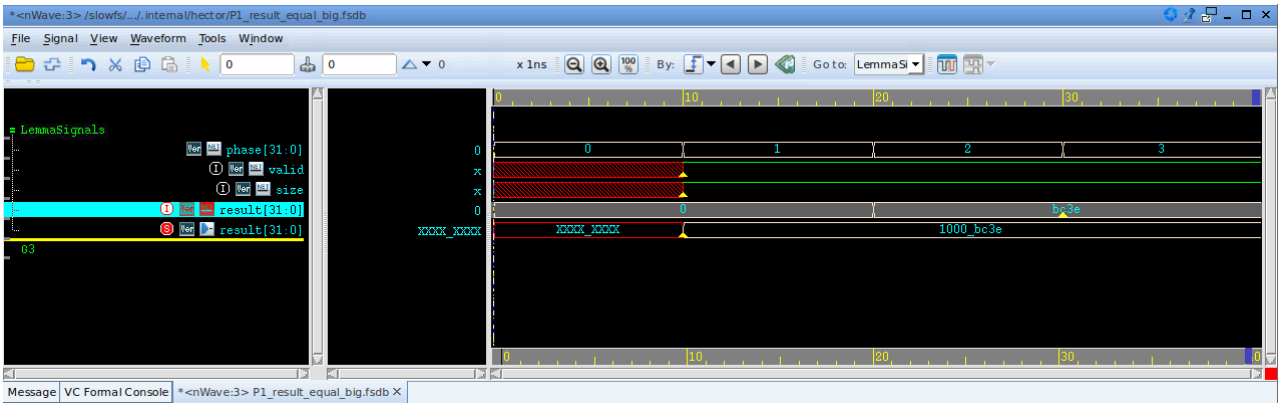


# Debug Falsified Lemmas to Fix RTL and Rerun the Proof

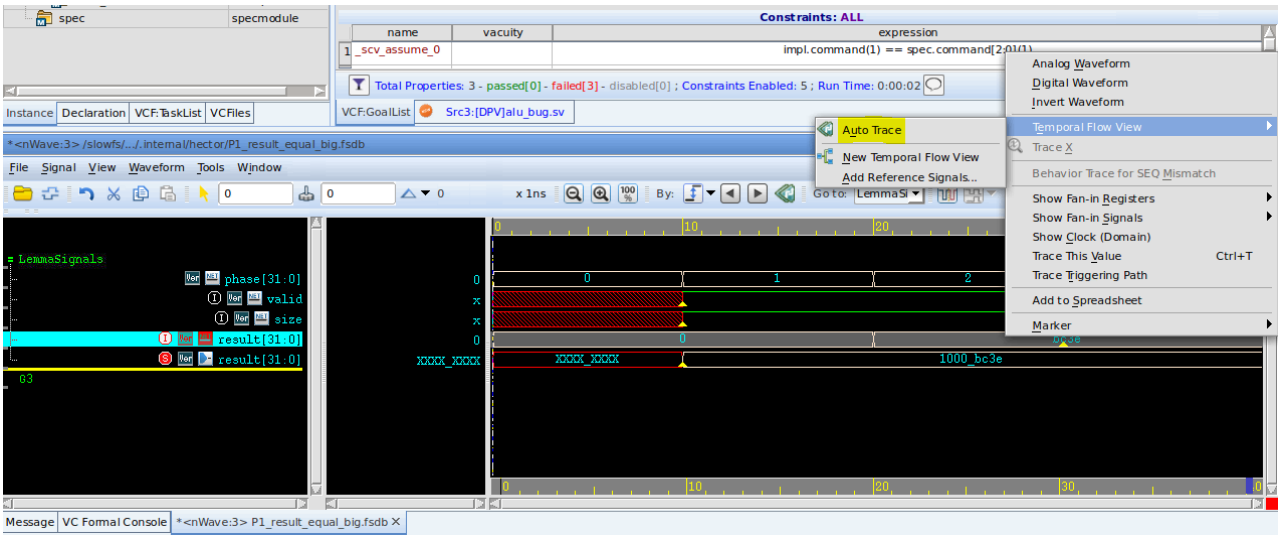
11. Debug the falsified lemmas. Let us look at the first one:



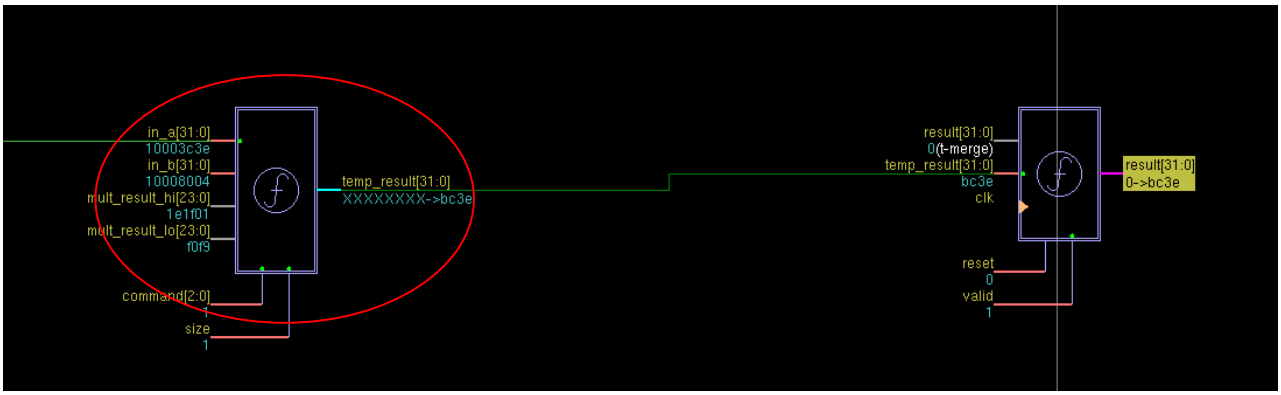

Double click on to open the waveform viewer. Notice that the RTL result mismatches the spec result.



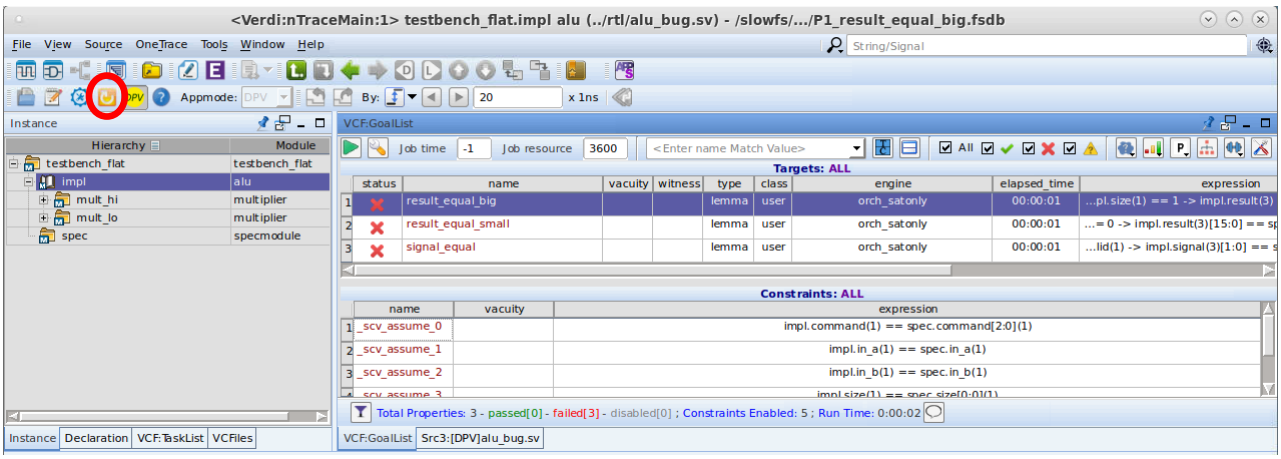

12. RMB on the value of RTL result then move to “Temporal Flow View” then select “Auto Trace”

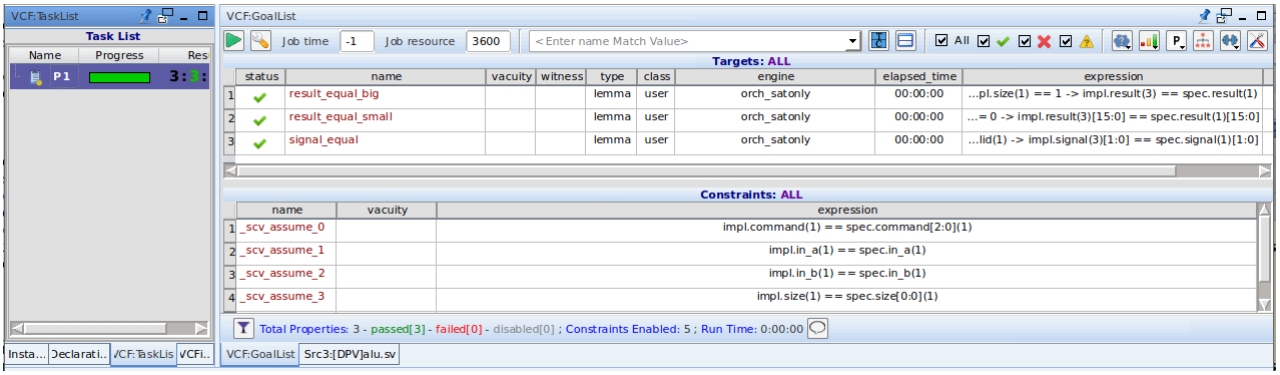



Track it back until finding the root point where there is unexpected result, then double click it to cross probe to the corresponding RTL to understand why the mismatch happens.




13. After RTL bug is fixed, rerun by clicking "restart" icon.




Make sure to repeat steps 8 - 10. All the lemmas should be proven as shown below.